

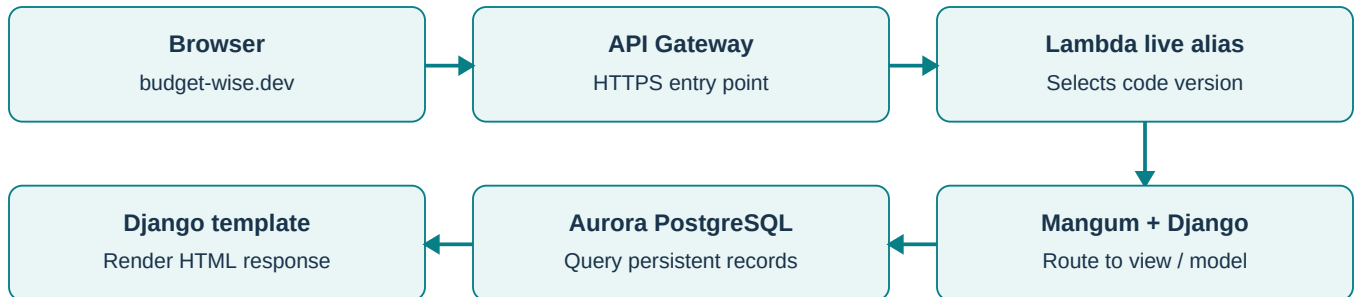
# BudgetWise | Architecture

Team 3 • Updated 05 October 2026 • Request flow and decisions

**Current state:** The live Lambda scaffold serves budget-wise.dev but / returns 404. Noah's model and migrations are merged. PR #33 has passing CI; the database-backed page and first automated deployment are still pending.

## Request flow: how a database-backed page will work

Cloudflare DNS resolves the domain to API Gateway; it does not run Django.



Database results return to Django. HTML returns through Lambda and API Gateway to the browser.

## Why these choices

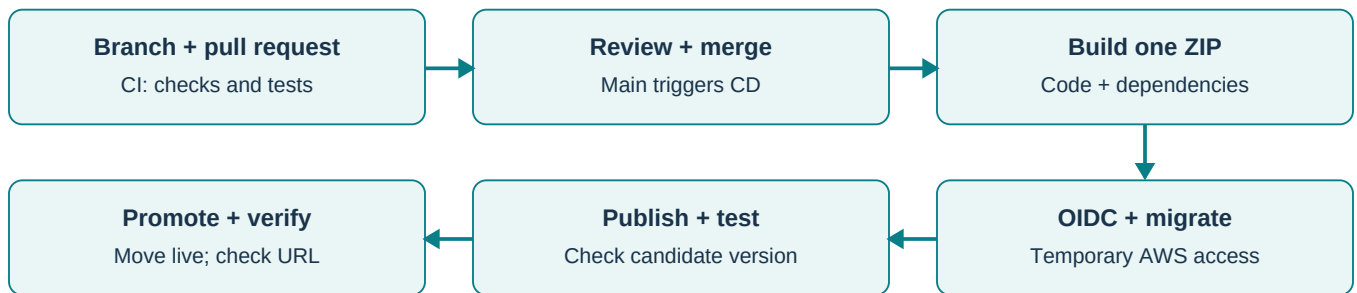
|                          |                                                                                                                                                                                                                                              |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Django</b>            | One Python project for views, templates, models and migrations. No separate hosted frontend is needed. API Gateway can serve ordinary pages; a separate public API is not required.                                                          |
| <b>Lambda + HTTP API</b> | Run Django on demand without managing a server or Elastic Beanstalk. Mangum translates AWS requests for Django. Cold starts and dependency packaging are tradeoffs.                                                                          |
| <b>Aurora PostgreSQL</b> | Keep deployed data independent of code releases. Serverless v2 express is configured for 0–1 ACU and five-minute idle auto-pause. End-to-end wake-up behavior still needs verification.                                                      |
| <b>IAM + TLS</b>         | The Lambda role supplies AWS credentials; our backend signs a fresh database token for each new connection and verifies TLS. The app user reads/writes records; the migration user changes the schema.                                       |
| <b>Networking</b>        | Lambda is outside a VPC and uses Aurora express's internet access gateway. No NAT gateway, load balancer or RDS Proxy. The existing AWS region is us-east-1.                                                                                 |
| <b>Secrets</b>           | Django reads Lambda environment variables for its private signing key and DB settings. GitHub stores deployment configuration and the DB_HOST secret. Secrets Manager is not currently used. No permanent AWS keys or DB password go in Git. |

**Data stays separate:** local SQLite persists in a local file; CI PostgreSQL is disposable; deployed Aurora retains app data. Closing DB connections allows idle pause without deleting records.

# BudgetWise | Development and delivery

One shared deployment; routine development and tests stay off AWS.

## Deployment flow



**Develop:** teammates use SQLite and README commands without individual AWS access. CI applies migrations and runs tests on temporary PostgreSQL 17, using no AWS credentials or Aurora credits.

**Build and deploy:** relevant main changes trigger CD; documentation-only edits skip it. Manual CD is available on main. The build job has no AWS access. It packages src/ and installed libraries once; library code belongs in the ZIP, not in Git.

**Authorize and check:** GitHub OIDC obtains temporary credentials for a role restricted to this repo's main branch. CD migrates Aurora from that ZIP, publishes a Lambda version, and checks its DB connection and homepage before moving live.

**Rollback:** a failed public check attempts to restore the previous alias. This rolls back code, **not migrations**; schema changes must support the previous release. ROOT\_STATUS is 404 now; change it to 200 when the homepage is added.

## Selected for later work - not implemented app features

|                         |                                                                                                                                                                                    |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Accounts / email</b> | Django auth for required account flows. Resend domain setup is complete; app email, sender/reply handling and reminder scheduling remain to be built.                              |
| <b>AI / imports</b>     | OpenAI insights with the professor-provided key; RapidOCR trial for receipts; Plaid Sandbox for test bank imports. Textract is only a fallback subject to eligibility/cost review. |
| <b>Uploads / assets</b> | Private S3 is planned for uploads. Static files stay in Git; their serving method remains undecided. Do not assume development static-file serving works on Lambda.                |

## What the team must remember

Django must enforce account access, record ownership and validation; AWS IAM does not replace those checks. Use fictional demo data until account isolation exists. Keep integration keys server-side.

Preserve the AWS Free Plan and credits: test locally, make only a few live checks, and review costs/eligibility before adding services or changing plans. Auto-pause and throttling are not zero-cost guarantees. CI/CD does not check billing-plan status.

**Code map:** src/lambda\_entry.py: HTTP adapter • src/budgetwise/db/aurora/: IAM tokens • src/transactions/: model/migrations • tests/: checks • .github/workflows/: CI/CD. README.md covers setup; docs/DEPLOYMENT.md covers release and rollback.

Evidence: repository 29ac427; PR #33 open with passing CI. Provider setup reflects this session's recorded configuration; no live AWS changes or billing checks were made for this update. Prepared with Codex assistance for team review.

# BudgetWise | Why these choices

Alternatives discussed, reasons selected, and tradeoffs accepted.

These choices fit our class project; they do not imply the alternatives cannot work.

## Cloudflare DNS

Instead of Route 53 registration: the account's Free Plan restriction and your no-upgrade requirement drove this choice. DNS can point to AWS independently. Tradeoff: manage two providers; registration does not create an email inbox.

## Lambda + Django

Instead of EC2/Lightsail or an always-running deployment: most of the semester will be idle, so on-demand execution fits the goal. Those servers offer a more conventional runtime but require server upkeep. Tradeoff: cold starts, packaging and execution limits. Cloudflare Workers was discussed, but Python-package and compute compatibility were untested.

## Django templates

Instead of a separate hosted frontend: build on Fabian's scaffold and keep the initial page, backend and models together. Tradeoff: the UI follows Django's rendering approach, and static-asset delivery still needs a decision.

## HTTP API

Instead of Function URLs, REST API or ALB: API Gateway provides the chosen HTTPS entry point and throttling without a load balancer. Function URLs were the simpler option; REST API offers more controls. Tradeoff: another service to configure and fewer controls than REST API; Django still enforces user permissions.

## Aurora PostgreSQL

Instead of provisioned RDS, DSQL or live SQLite: relational data and familiar Django migrations fit PostgreSQL; Aurora's idle pause matches the usage goal. DSQL needed a compatibility investigation; live SQLite needed shared persistent storage. Tradeoff: wake-up latency, connection management and storage usage remain. A single self-managed server was discussed but gave the team more database maintenance work.

## IAM + express gateway

Instead of a stored DB password and conventional VPC/NAT path: use role-based access while retaining outbound access to future external APIs. Tradeoff: token generation, TLS and matching SQL grants need explicit setup. Runtime and migration roles stay separate to limit schema access.

## Local SQLite + CI PostgreSQL

Instead of giving everyone live Aurora access: teammates can work offline from AWS, while CI catches PostgreSQL differences. Tradeoff: local data is not shared, and passing CI still cannot prove the deployed IAM/network path.

## GitHub Actions + OIDC

Instead of manual ZIP uploads and permanent AWS keys: reuse the team's PR workflow and issue temporary deployment credentials. Tradeoff: maintain workflow permissions and release checks. Automatic main deployment was selected during implementation; manual dispatch remains available.

## Django auth + Resend

Instead of Cognito: Django gives control over the required verification, reset and logout flows; this was not a claim of meaningful cost savings. Instead of SES/SNS: Resend was selected for straightforward app email; SES required sandbox/access consideration and SNS targeted subscribed alerts. Tradeoff: we must implement/test account flows and manage a separate email provider.

## Secrets Manager: planned

Likely choice for future API keys; not implemented yet. Central management avoids copying keys through deployments, but adds retrieval/caching code, permissions and service usage. Lambda environment variables remain the simpler alternative to decide on. The current Django signing key uses Lambda environment configuration. GitHub secrets alone are not available to a running Lambda.

## Later integrations

OpenAI replaced Gemini with a professor-provided key. RapidOCR avoids an initial paid OCR API, at the cost of accuracy/package-size testing; Textract stays conditional on plan eligibility. Plaid Sandbox uses test institutions rather than real-bank production access. S3 was selected for files needing persistence, with retention/deletion rules still to define.

Sources: "Compare BudgetWise hosting options" and "Configure BudgetWise domain and Res"; current implementation and your API-key clarification. Historical rationale, not refreshed pricing or quotas.